

Core Components Assessment

GCSE Computer Science · Python Lab · Archbishop McGrath Catholic High School

Bobby · Class not entered · 10CS · 2026-09-24



32 / 56 marks · 57.1% · 24 not yet achieved · 0 unmarked
 Developing · Teacher-reviewed classroom scheme

Developing: 25% (14 marks) · Secure: 60% (34 marks) · Mastery: 80% (45 marks). These are classroom checkpoints, not GCSE grades. Unmarked criteria have not been treated as incorrect.

Assessment breakdown

Task	Achieved	Not yet	Unmarked
A · Knowledge & tracing	6 / 9	3	0
B · Debug lab	5 / 6	1	0
C1 · Merchandise order	9 / 12	3	0
C2 · Priority delivery	6 / 8	2	0
C3 · Booking repair	6 / 9	3	0
C4 · Tournament budget	0 / 12	12	0

A message from your teacher

Bobby, this is a solid result and shows that you understand the main programming concepts covered.

To progress further, you now need to focus on the precision and overall quality of your code. Take greater care to include useful comments, make sure your outputs are clear and meaningful rather than just displaying raw numbers, and use f-strings confidently to add context.

The core understanding is there, so your next step is to refine and polish your code so that it is clearer, more accurate and more professional.

WWW · What went well

Type casting · `int()` and `float()`

You selected an appropriate numerical conversion in the recorded work.

Evidence: C1.3, C1.4.

Calculations and tracing

You selected appropriate operators and linked stored values to calculate results.

Evidence: C1.5, C1.6.

Variables

You used variables to store inputs or calculation results and refer to them later.

Evidence: C1.1, C1.2.

Inputs · `input()`

You collected and stored the values needed by the program.

Evidence: C1.1, C1.2.

EBI · Even better if

Naming conventions and comments

Choose meaningful, valid variable names and use one naming convention consistently. Add comments that explain a section's purpose.

Evidence: C1.11, C2.8.

F-strings and formatting

Use `f` before the quotation marks and put variables inside `{braces}`. Use `:.2f` for monetary output when the task requires two decimal places.

Evidence: C1.10, C2.7.

Outputs · print()

Use print() to display each required result. Distinguish literal text in quotes from a variable's stored value.

Evidence: C1.9, C1.10.

Individual mark checklist

Each criterion shows its maximum mark. "Not yet" is a missed mark; "Unmarked" awaits a decision. This is a proposed teacher scheme based on the workbook's questions and requirements, not an official exam-board scheme.

A1 · Question 01 · 1 / 1

Criterion	What was required	Decision
A1.1	Give the correct answer: Boolean.	1 / 1

A2 · Question 02 · 1 / 1

Criterion	What was required	Decision
A2.1	Give the correct answer: firstScore.	1 / 1

A3 · Question 03 · 1 / 1

Criterion	What was required	Decision
A3.1	Give the correct answer: age = int(input("Age: ")).	1 / 1

A4 · Question 04 · 1 / 1

Criterion	What was required	Decision
A4.1	Give the correct answer: 3.	1 / 1

Teacher note: Take care to only select one answer!

A5 · Question 05 · 0 / 1

Criterion	What was required	Decision
A5.1	Give the correct answer: print(workerName).	0 / 1

A6 · Question 06 · 1 / 1

Criterion	What was required	Decision
A6.1	Give the correct answer: 4.	1 / 1

A7 · Question 07 · 1 / 1

Criterion	What was required	Decision
A7.1	Give the correct answer: Inspector: Sam.	1 / 1

A8 · Question 08 · 0 / 2

Criterion	What was required	Decision
A8.1	Explain that a syntax error breaks Python's language rules and prevents the affected code from running.	0 / 1
A8.2	Explain that a logic error allows a program to run but produces the wrong result.	0 / 1

B1 · Debug 01 · Merchandise order · 3 / 3

Criterion	What was required	Decision
B1.1	Convert the quantity input to an integer before using it in arithmetic.	1 / 1

Criterion	What was required	Decision
B1.2	Use quantity × unitPrice to calculate the total.	1 / 1
B1.3	Use an f-string to combine the output text with the total variable.	1 / 1

B2 · Debug 02 · Time converter · 2 / 3

Criterion	What was required	Decision
B2.1	Use integer division by 60 to calculate complete hours.	1 / 1
B2.2	Use remainder after division by 60 to calculate the remaining minutes.	0 / 1
B2.3	Produce the correct labelled result: 2 hours and 22 minutes.	1 / 1

Teacher note: use % for remainder calculations

C1 · Practical 01 · Gaming merchandise order · 9 / 12

Criterion	What was required	Decision
C1.1	Collect and store the customer's name.	1 / 1
C1.2	Collect and store the product name.	1 / 1
C1.3	Collect quantity and convert it to an integer.	1 / 1
C1.4	Collect unit price and convert it to a float.	1 / 1
C1.5	Calculate subtotal as quantity × unit price.	1 / 1
C1.6	Use a fixed delivery charge of £2.50.	1 / 1
C1.7	Calculate total as subtotal + delivery.	1 / 1
C1.8	Identify the customer, product and quantity in the order summary.	1 / 1
C1.9	Display labelled subtotal, delivery and total values.	0 / 1
C1.10	Format every monetary amount to two decimal places.	0 / 1
C1.11	Add useful section comments.	0 / 1
C1.12	Use meaningful variable names that make the program easy to follow.	1 / 1

Teacher note: Make sure you include labels with the variable values. Your output values aren't clear to the end user here.

Use f strings

C2 · Practical 02 · Priority delivery upgrade · 6 / 8

Criterion	What was required	Decision
C2.1	Retain the existing item, quantity and unit-price inputs with correct types.	1 / 1
C2.2	Retain the correct subtotal calculation.	1 / 1
C2.3	Store a fixed priority-delivery charge of £6.00.	1 / 1
C2.4	Calculate the updated total as subtotal + priority delivery.	1 / 1
C2.5	Display a PRIORITY ORDER heading.	1 / 1
C2.6	Show the item and a clear subtotal, delivery and total breakdown.	1 / 1
C2.7	Display all monetary values to two decimal places.	0 / 1
C2.8	Add a useful comment explaining the new section.	0 / 1

C3 · Practical 03 · Gaming lounge booking repair · 6 / 9

Criterion	What was required	Decision
C3.1	Convert the number of teams to an integer.	1 / 1
C3.2	Convert players per team to an integer.	1 / 1
C3.3	Convert price per player to a float.	0 / 1
C3.4	Calculate seats as teams × players per team.	1 / 1
C3.5	Calculate total price as seats × price per player.	1 / 1
C3.6	Repair the seat output so it displays without a text/number type error.	1 / 1
C3.7	Repair the total-price output so it displays without a type error.	1 / 1
C3.8	Display the total price as money to two decimal places.	0 / 1
C3.9	Add useful comments explaining the repaired program.	0 / 1

Teacher note: Prices can contain decimals, these need to be floats

C4 · Practical 04 · Sports tournament budget · 0 / 12

Criterion	What was required	Decision
C4.1	Collect and store the tournament name.	0 / 1
C4.2	Collect team count and players per team as appropriate whole numbers.	0 / 1
C4.3	Collect entry price per player as a decimal-capable number.	0 / 1
C4.4	Collect venue hire and officials cost as decimal-capable numbers.	0 / 1
C4.5	Calculate total players as teams × players per team.	0 / 1
C4.6	Calculate entry income as total players × entry price.	0 / 1
C4.7	Calculate total costs as venue hire + officials cost.	0 / 1
C4.8	Calculate the expected balance as income – total costs.	0 / 1
C4.9	Display a complete, clearly labelled budget summary.	0 / 1
C4.10	Format the monetary values to two decimal places.	0 / 1
C4.11	Use purposeful comments to organise the budget program.	0 / 1
C4.12	Use meaningful identifiers throughout the budget program.	0 / 1

Progress rubric

Developing means some requirements achieved; Secure means the key concepts achieved; Mastery means all checklist requirements for that skill achieved. The green tick shows the selected level. Skill levels are separate from the overall checkpoint badge.

Skill	Developing	Secure	Mastery
Inputs · input() Mastery	I can collect a value from the user and store it in a variable.	I can use clear prompts to collect and store the main information a program needs.	✓ I can collect and store all the information needed for each program, including names, quantities, prices and costs.
Outputs · print() Secure	I can display text and the value stored in a variable.	✓ I can display the main results with clear labels so the user can understand them.	I can produce complete, clearly labelled summaries that show every requested detail and result, with money formatted where required.
Variables Mastery	I can create variables and store values in them.	I can store inputs and calculation results in variables, then reuse those values in calculations and output.	✓ I can assign, update and reuse values accurately throughout my programs, and correctly trace how variable values change.

Skill	Developing	Secure	Mastery
Naming conventions and comments Developing	✓ I can use valid variable names and add a simple comment.	I can choose meaningful names and use comments to identify the main sections of my code.	I can make my programs easy to follow, using meaningful names throughout and useful comments for the sections, changes and repairs requested.
Type casting · int() and float() Developing	✓ I can convert an input into a number using int() or float().	I can use int() for whole-number counts and float() for prices so the values can be used in calculations.	I can apply suitable conversions to every numerical input, handling whole-number quantities and decimal prices and costs correctly.
Calculations and tracing Secure	I can carry out simple arithmetic and follow a short sequence of calculations.	✓ I can choose the correct operators to calculate quantities, subtotals, totals and remainders.	I can build and trace the required calculations accurately from start to finish, including the time conversion and the tournament's income, costs and final balance.
F-strings and formatting Developing	✓ I can recognise an f-string and use one to include a variable in a simple message.	I can combine text and variable values correctly using the f prefix and braces {}.	I can correctly predict and produce f-string output, and display monetary values to two decimal places wherever the task requires it.
Python knowledge and error types Secure	I can recognise basic Python terms, including Boolean values and programming errors.	✓ I can identify Boolean values and distinguish syntax errors from logic errors, although my description of their effects may be incomplete.	I can correctly identify Boolean values and describe how syntax errors break Python's rules and prevent code running, while logic errors produce incorrect results.
Debugging and repair Secure	I can identify and correct a simple mistake in supplied code.	✓ I can repair common mistakes in input types, calculation operators and output statements.	I can fully repair the supplied programs so their inputs, calculations and outputs work together correctly, while retaining the code that already works.

DIRT · Improve your work

[] **Your improvement task:** Improve and complete all sections in this assessment.
Use the support materials in the workbooks and unit 1 digital textbook to support you.

Add comments to all your code to explain the key concepts and sections.
Use f-strings with outputs to give data values meaning.

What did you change?

You can find my improved work/DIRT task at

DIRT completion does not change the original assessment mark. Keep pupil information private.